

Getting Started with Python

Basic Concepts and Code Examples for Beginners

1. Hello, World!

This is the first Python program most beginners write. It prints text to the screen:

```
print("Hello, World!")
```

When you run it, you'll see:

```
Hello, World!
```

2. Variables and Types

A variable stores a value you can use later:

```
x = 5          # integer
name = "Sam"   # string
pi = 3.14      # float
```

Python figures out the type automatically. For example, here `x` is an integer, and `name` is text (a string).

3. Lists

A list is a collection of values:

```
numbers = [1, 2, 3, 4]
print(numbers)
```

You can add lists together:

```
a = [1, 2]
b = [3, 4]
print(a + b)  # [1, 2, 3, 4]
```

Lists can contain any type of item.

4. Basic Operators

Python lets you do math and combine values:

```
print(2 + 3)    # addition
print(5 * 2)    # multiplication
print(10 % 3)   # remainder
```

You can also use operators with strings and lists:

```
print("hi" * 3)      # hi repeated 3 times
print([1] * 4)       # [1, 1, 1, 1]
```

Adding lists joins them.

5. String Formatting

You can put variables into text easily:

```
name = "Alex"
age = 20
print(f"{name} is {age} years old.")
```

This prints the values inside a sentence using an f-string.

6. Basic String Operations

Strings are text, and you can work with them:

```
s = "hello"
print(s.upper())    # HELLO
print(s[1:4])       # "ell" (slice)
```

Strings support indexing and many built-in methods.

7. Conditions

Use if, elif, and else to make decisions:

```
x = 10

if x > 5:
    print("x is big")
elif x == 5:
    print("x is five")
```

```
else:  
    print("x is small")
```

Python checks each condition top to bottom and runs the first true block.

8. Loops

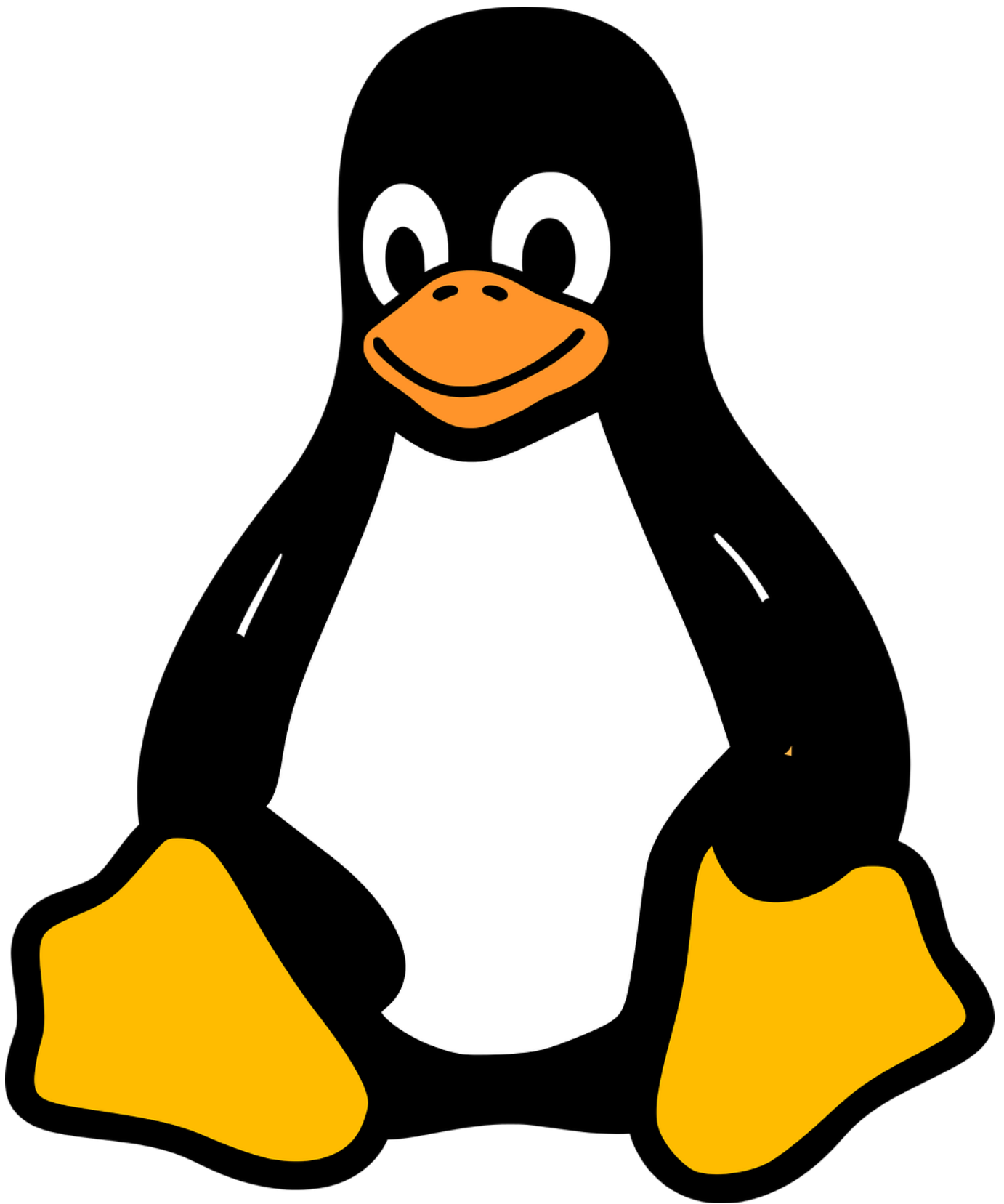
Loops let you repeat code. For loop — for each item in something:

```
for i in range(5):  
    print(i)
```

While loop — repeat while a condition is true:

```
count = 0  
while count < 3:  
    print("Hello")  
    count += 1
```

Loops are great for repeating tasks.



What is Linux?

Just like Windows, iOS, and Mac OS, Linux is an operating system. In fact, one of the most popular platforms on the planet, Android, is powered by the Linux operating system. An operating system is software that manages all of the hardware resources associated with your desktop or laptop. To put it simply, the operating system manages the communication between your software and your hardware. Without the operating system (OS), the software wouldn't function.

The Linux operating system comprises several different pieces:

Bootloader – The software that manages the boot process of your computer. **Kernel** – This is the one piece of the whole that is actually called 'Linux'. The kernel is the core of the system and manages the CPU, memory, and peripheral devices. The kernel is the lowest level of the OS. **Init system** – This is a sub-system that bootstraps the user space and is charged with controlling daemons. **Daemons** – These are background services (printing, sound, scheduling, etc.) **Graphical server** – This is the sub-system that displays the graphics on your monitor. **Desktop environment** – This is the piece that the users actually interact with. **Applications** – Desktop environments do not offer the full array of apps. Just like Windows and macOS, Linux offers thousands upon thousands of high-quality software titles that can be easily found and installed.

Why use Linux?

Linux has evolved into one of the most reliable computer ecosystems on the planet. Combine that reliability with zero cost of entry and you have the perfect solution for a desktop platform.

Linux is also distributed under an open source license.

Open source follows these key tenets:

1. The freedom to run the program, for any purpose.
2. The freedom to study how the program works, and change it to make it do what you wish.
3. The freedom to redistribute copies so you can help your neighbor.
4. The freedom to distribute copies of your modified versions to others.
5. These points are crucial to understanding the community that works together to create the Linux platform. Without a doubt, Linux is an operating system that is "by the people, for the people". These tenets are also a main factor in why many people choose Linux. It's about freedom and freedom of use and freedom of choice.

What is a "distribution?"

Linux has a number of different versions to suit any type of user. From new users to hard-core users, you'll find a "flavor" of Linux to match your needs. These versions are called distributions (or, in the short form, "distros"). Nearly every distribution of Linux can be downloaded for free, burned onto disk (or USB thumb drive), and installed (on as many machines as you like).

Popular Linux distributions include:

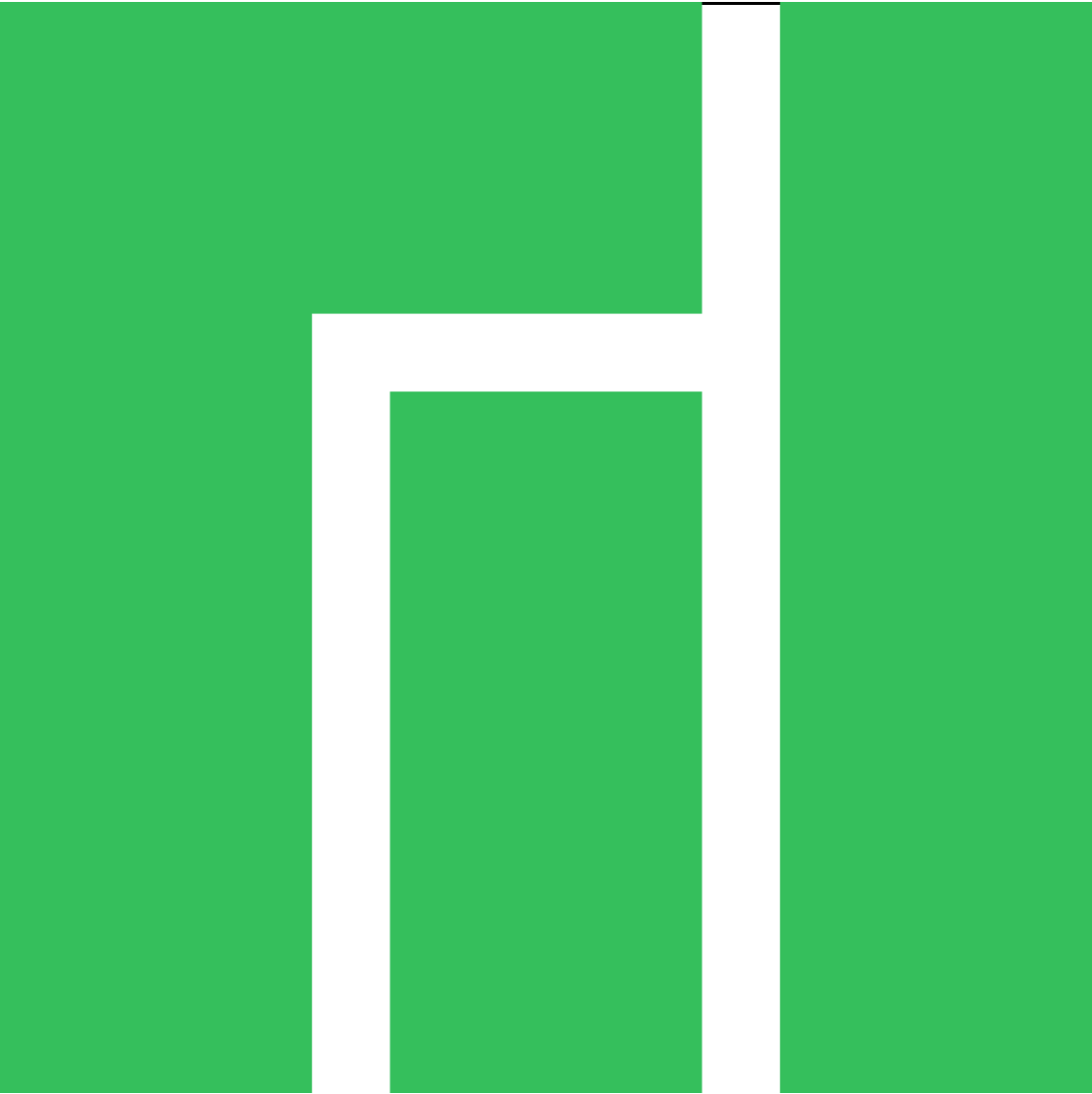
Popular Linux distributions include:



LINUX MINT

MANJARO

Popular Linux distributions include:



DEBIAN

Popular Linux distributions include:



UBUNTU



ANTERGOS

Popular Linux distributions include:

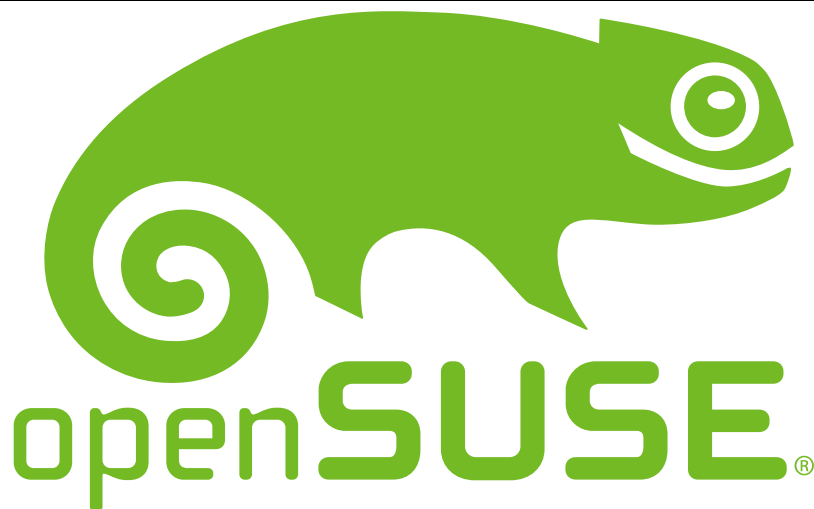
SOLUS

FEDORA

Popular Linux distributions include:



ELEMENTARY
OS



OPENSUSE

Which distribution is right for you?

Which distribution you use will depend on the answer to three simple questions:

1. How skilled of a computer user are you?
2. Do you prefer a modern or a standard desktop interface?

3. Server or desktop? If your computer skills are fairly basic, you'll want to stick with a newbie-friendly distribution such as Linux Mint, Ubuntu (Figure 3), Elementary OS or Deepin. If your skill set extends into the above-average range, you could go with a distribution like Debian or Fedora. If, however, you've pretty much mastered the craft of computer and system administration, use a distribution like Gentoo. If you really want a challenge, you can build your very own Linux distribution, with the help of Linux From Scratch.
4. If you're looking for a server-only distribution, you will also want to decide if you need a desktop interface, or if you want to do this via command-line only. The Ubuntu Server does not install a GUI interface. This means two things your server won't be bogged down loading graphics and you'll need to have a solid understanding of the Linux command line. However, you can install a GUI package on top of the Ubuntu Server with a single command like `sudo apt-get install ubuntu-desktop`. System administrators will also want to view a distribution with regards to features. Do you want a server-specific distribution that will offer you, out of the box, everything you need for your server? If so, CentOS might be the best choice. Or, do you want to take a desktop distribution and add the pieces as you need them? If so, Debian or Ubuntu Linux might serve you well.

Installing Linux

Linux offers one of the easiest installations of all operating systems. In fact, most versions of Linux offer what is called a Live distribution, which means you run the operating system from either a CD/DVD or USB flash drive without making any changes to your hard drive. You get the full functionality without having to commit to the installation.

Typically, the installation wizards walk you through the process with the following steps (We'll illustrate the installation of Ubuntu Linux):

1. Preparation: Make sure your machine meets the requirements for installation. This also may ask you if you want to install third-party software (such as plugins for MP3 playback, video codecs, and more).
2. Wireless setup (if necessary): If you are using a laptop (or machine with wireless), you'll need to connect to the network, in order to download third-party software and updates.
3. Hard drive allocation (Figure 4): This step allows you to select how you want the operating system to be installed. Are you going to install Linux alongside another operating system (called "dual booting"), use the entire hard drive, upgrade an existing Linux installation, or install over an existing version of Linux.
4. Location: Select your location from the map.
5. Keyboard layout: Select the keyboard for your system.
6. User setup: Set up your username and password.
7. That's it. Once the system has completed the installation, reboot and you're ready to go. For a more in-depth guide to installing Linux, take a look at "How to Install and Try Linux the Absolutely Easiest and Safest Way" or download the Linux Foundation's PDF guide for Linux installation.

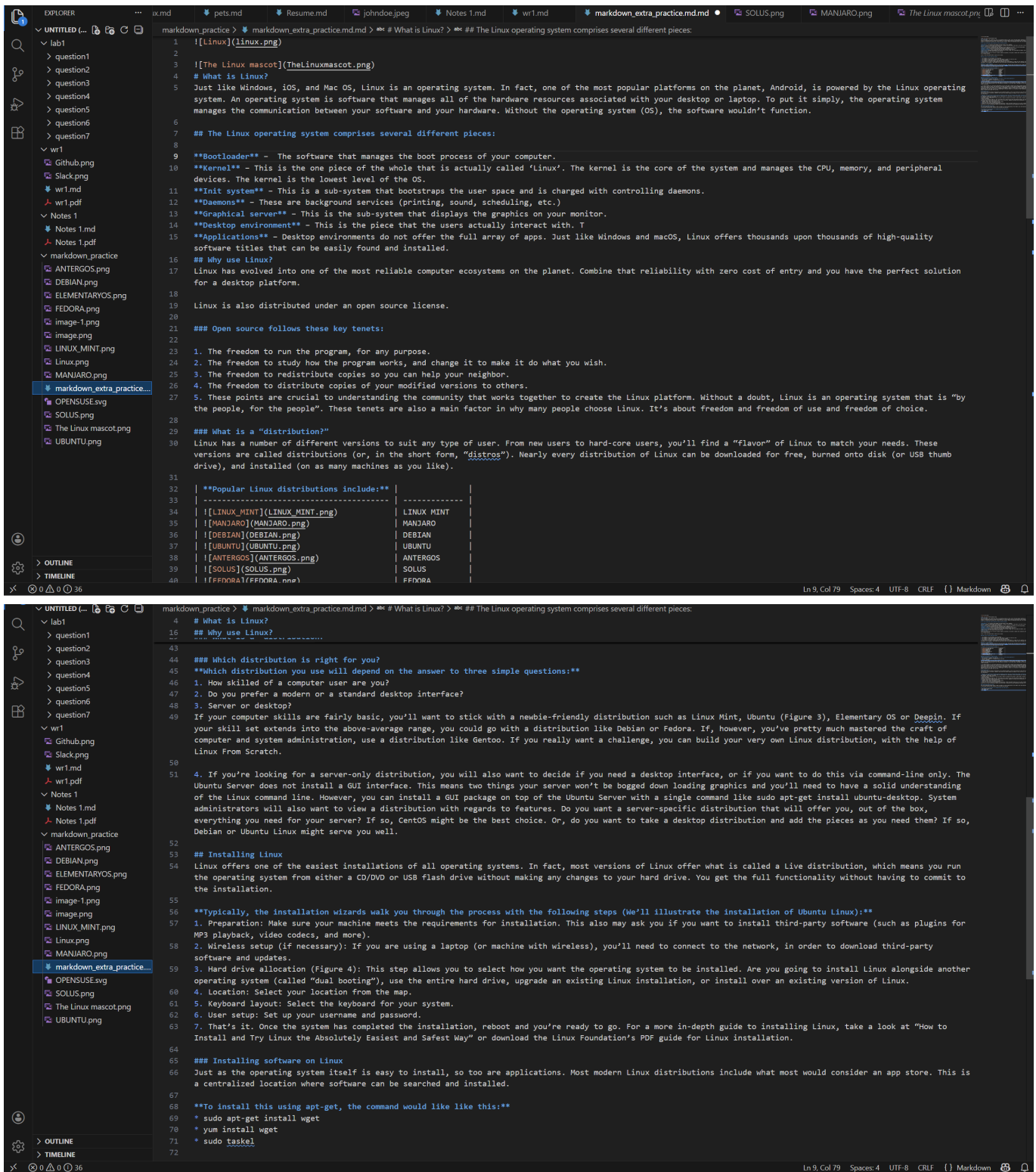
Installing software on Linux

Just as the operating system itself is easy to install, so too are applications. Most modern Linux distributions include what most would consider an app store. This is a centralized location where software can be searched and installed.

To install this using apt-get, the command would look like this:

- sudo apt-get install wget
- yum install wget
- sudo taskel

Desktop environment screenshots



Notes 1

What is Markdown?

Markdown is a lightweight markup language that you can use to add formatting elements to plaintext text documents. Created by John Gruber in 2004, Markdown is now one of the world's most popular markup language.

What is Git?

Git is a version control system that helps you track changes in your files over time. In simple terms, Git is like a save history button for your projects.

What is GitHub?

GitHub is an online platform where people store, manage, and work together on code and projects. In simple terms, it's like Google Drive for code, but with powerful tools for tracking changes and collaborating.

What is Slack?

Slack is a communication and collaboration app used by teams to talk and work together in one place. In simple terms, Slack is like WhatsApp or Discord for school and work, but more organized.

Lecture 2 Introduction to Linux Notes

1. **What is an Operating System?** An Operating System (OS) is the main software that runs your computer. It acts like a manager between you and the computer's hardware. It helps you run programs, manage files, and use devices like your keyboard and printer. Simple example: Windows, macOS, and Linux are all operating systems.
2. **What is a kernel?** The kernel is the heart of an operating system. It's the core part that directly talks to your computer's hardware (like the CPU, memory, and hard drive). Think of it as the "brain" that controls everything behind the scenes.

3. **Which other parts aside from the kernel identify an OS?** Besides the kernel, an OS includes:

User Interface - What you see and interact with (desktop, icons, menus)
System Libraries - Collections of code that programs use to work
System Utilities - Basic tools like file managers, text editors, and settings
Application Software - Programs you use daily (web browsers, office apps)

4. **What is linux and linux distribution?**

A Linux distribution is a complete operating system package that includes:

The Linux kernel
Software applications
A desktop environment
Installation tools

Examples of distributions: Ubuntu, Fedora, Debian, Mint

5. **List at least 4 linux characteristics:** Free and Open Source - Anyone can use, view, and modify the code without paying
Secure - Less vulnerable to viruses and malware compared to other systems
Stable - Runs for long periods without crashing or needing restarts
Flexible - Can be customized for different uses (servers, phones, computers)

6. **What is Debian?**

Debian is one of the oldest and most popular Linux distributions. It started in 1993 and is known for being:

Very stable and reliable
Completely free
Community-driven (run by volunteers, not a company)

7. **List and define the different types of licensing agreements**

- Proprietary License

You pay for the software
You cannot view or modify the source code
Example: Microsoft Windows, Adobe Photoshop

- Free and Open Source License (FOSS)

Free to use, modify, and share
Source code is available to everyone
Example: Linux, Firefox, LibreOffice

- Shareware

Free to try for a limited time
Must pay to continue using after trial period
Example: WinRAR

- Freeware

Free to use forever But you cannot see or modify the source code Example: Adobe Acrobat Reader

8. What is Free Software? Define the 4 freedoms.

Free Software means software that respects users' freedom and community. "Free" refers to freedom.

The 4 Freedoms:

Freedom 0: Use the freedom to run the program for any purpose, anywhere, anytime

Freedom 1: Study the freedom to study how the program works and see its source code

Freedom 2: Share the freedom to make copies and distribute them to help others

Freedom 3: Improve the freedom to modify the program and share your improvements with the community

9. What is virtualization?

Virtualization is technology that lets you run multiple "virtual computers" on one physical computer.

Answers to the following questions:

What is a graphical user interface (GUI)?

A GUI is a way to interact with a computer using windows, icons, menus, and a mouse instead of typing commands. It is user-friendly and visual.

What is a desktop environment?

A desktop environment is the complete graphical workspace that includes the window manager, panels, menus, icons, and settings.

What is the command line interface (CLI)?

The CLI is a text-based interface where users type commands to interact with the system directly.

How do I access the command line interface (CLI)?

You can access the CLI by:

Opening terminal from the desktop menu

Pressing Ctrl + Alt + T

Switching to a virtual console

What is a virtual console?

A virtual console is a text-only login screen that runs without the graphical interface.

What is a terminal emulator?

A terminal emulator is a graphical program that lets you use the command line inside the desktop environment.

What is bash?

Bash (Bourne Again Shell) is the most common Linux shell. It interprets commands typed by the user and executes them.

What is the shell prompt?

The shell prompt is the text that appears when the shell is ready to accept commands.

Definition, usage, and examples of the following commands:

1. **clear** Definition: Clears the terminal screen Usage: Clean up the display


```
clear
```

2. **echo** Definition: Prints text or variables to the terminal Usage: Display messages or output values

```
echo "Hello World"
```

3. **date** Definition: Displays or sets the system date and time Usage: Show current date/time

```
date  
date --rfc-3339=ns
```

4. **free** Definition: Displays memory usage Usage: Check free and used RAM

```
free -h
```

5. **uname** Definition: Shows system and kernel information Usage: Identify OS and kernel details

```
uname -a
```

6. **history** Definition: Shows previously run commands Usage: Review command history

```
history
```

7. **man** Definition: Displays the manual page for a command Usage: Learn command options

```
man ls
```

8. **tldr** Definition: Simplified help pages with examples Usage: Quick command reference

```
tldr tar
```

9. **cheat** Definition: Community cheat sheets for commands Usage: Learn commands quickly

```
cheat docker
```

10. **hostname** Definition: Displays or sets the system hostname Usage: Identify machine name

```
hostname
```

11. **df** Definition: Shows disk space usage of filesystems Usage: Check available storage

```
df -h
```

12. **du** Definition: Displays disk usage of files and directories Usage: Find large folders

```
du -sh *
```

13. **figlet** Definition: Creates large ASCII text banners Usage: Decorative text output

```
figlet Linux
```

What to include in Notes 4

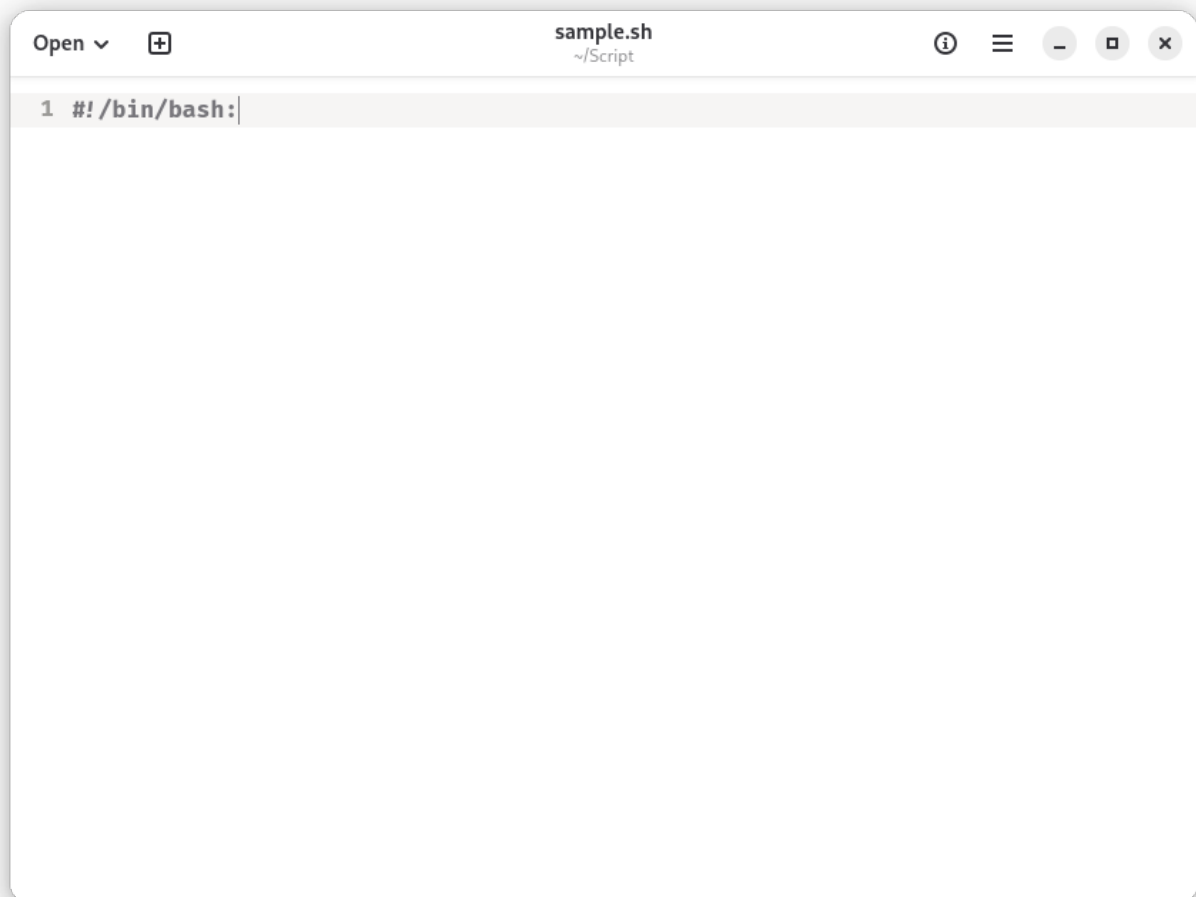
Answer the following questions:

1. How to install and remove software using the APT command

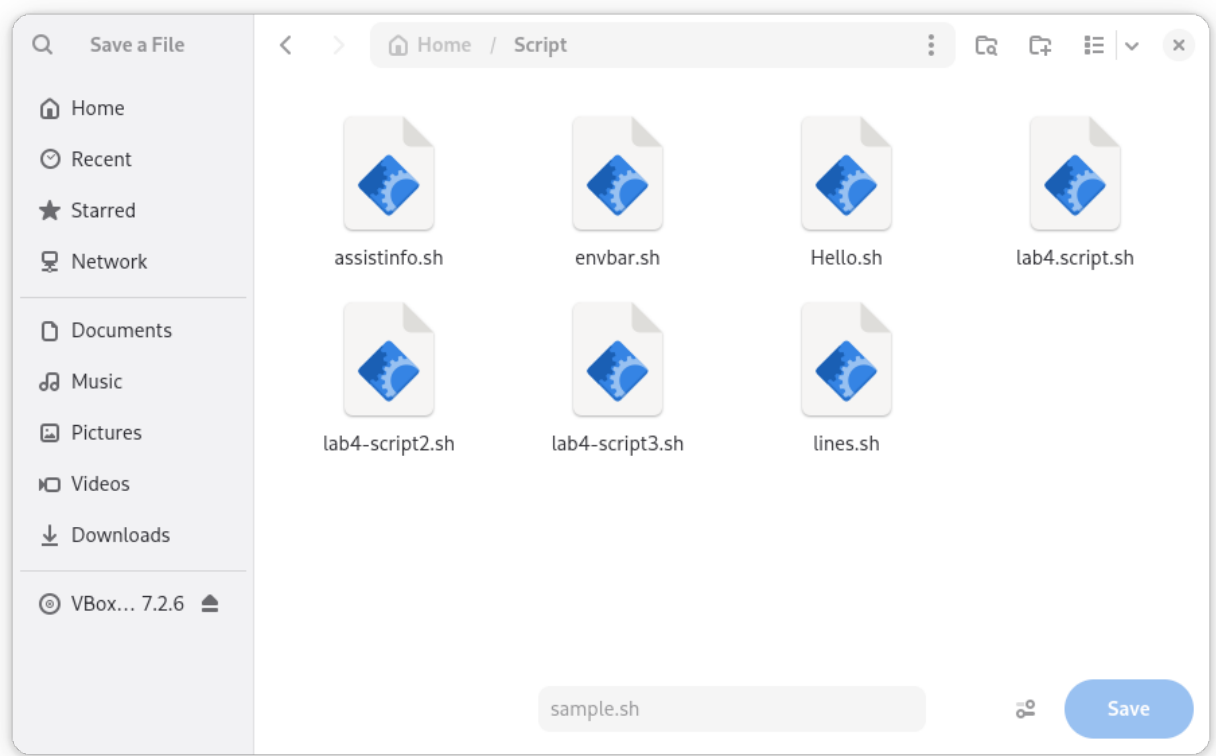
- Installing packages: `sudo apt install +package`
- Removing packages: `sudo apt remove +package` `sudo apt purge +package`

2. How to create a shell script step by step including screenshots and how to run it. Try to be as detailed as possible.

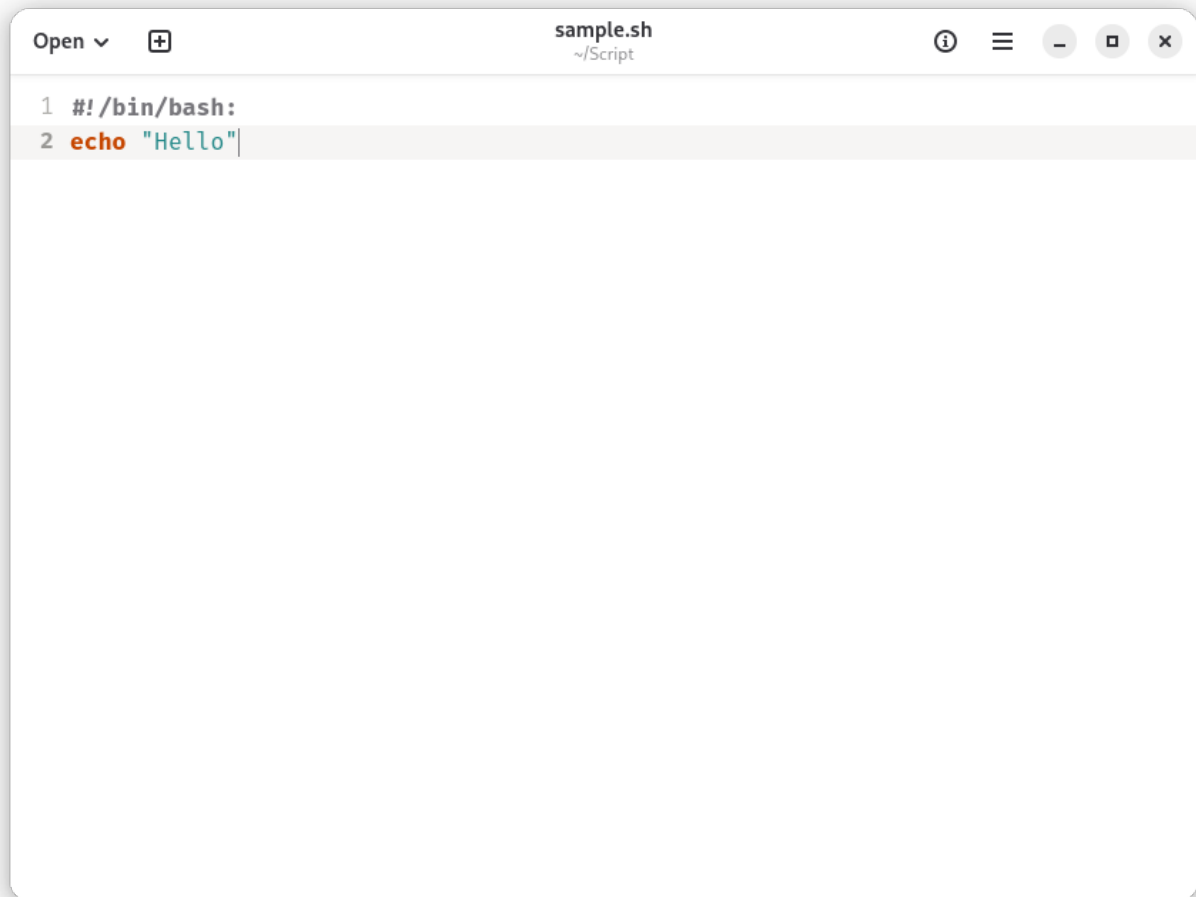
step 1: open the text editor and type the declaration statement: `#!/bin/bash:`



Step 2: save the file on your script folder with the extension `.sh`. (save it in a way you can remember because Linux is cases sensitive.)

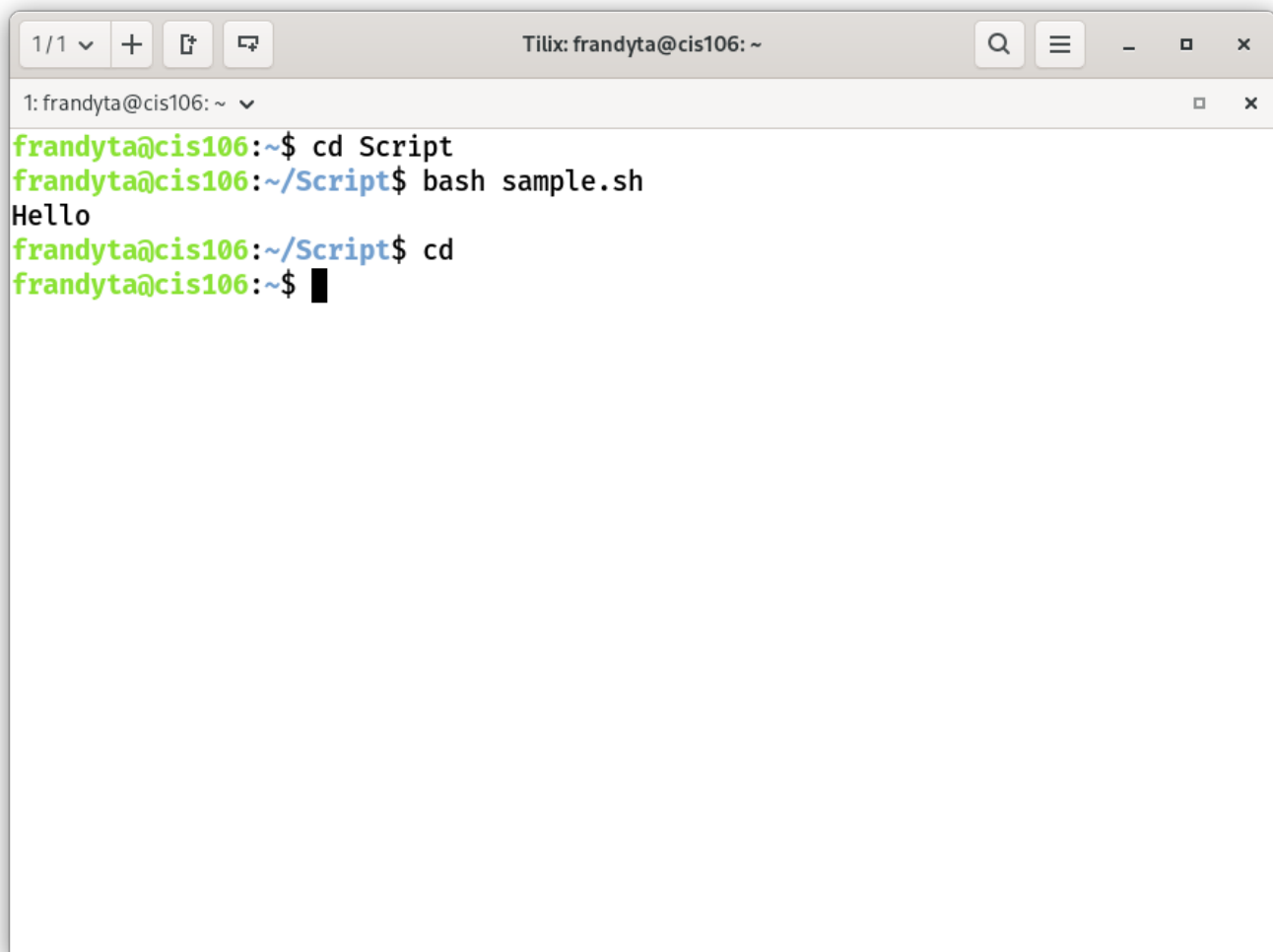


Step 3: Start typing, for example `:echo "Hello"`



```
1 #!/bin/bash:  
2 echo "Hello"
```

Step 4: Open your Terminal. Use the command: `cd scripts/`, then Run the script using this command: `bash sample.sh` and it will display "Hello".



A terminal window titled "Tilix: frandyta@cis106: ~" with standard window controls. The terminal shows a sequence of commands and their outputs:

```
1: frandyta@cis106: ~  
frandyta@cis106:~$ cd Script  
frandyta@cis106:~/Script$ bash sample.sh  
Hello  
frandyta@cis106:~/Script$ cd  
frandyta@cis106:~$
```

Notes 5

For each of the following commands include: Definition, Usage/Formula, and Examples.

ls: The ls command is used to list the files and folders in a directory.

Usage / Formula:

```
ls
```

Examples:

```
- ls
- ls Documents
- ls -l**
```

pwd The pwd command shows the current working directory (where you are in the system).

Usage / Formula:

```
pwd
/home/student/Documents
```

cd The cd command is used to change directories (move between folders).

Usage / Formula:

```
cd [directory_name]
```

Examples:

```
- cd Documents
- cd ..
- cd /home/student
```

Include answers to the following questions:

1. **What is a variable?** A variable is a name that stores a value. The value can change.

2. **How do I use a variable?** You assign a value and then use it later. **Example:**

```
name=John  
echo $name
```

3. **What is an environment variable?** An environment variable is a variable that is already set by the system and used by programs. **Example:**

```
echo $HOME
```

4. **What is a user defined variable?** A user defined variable is a variable created by the user. **Example:**

```
age=21  
echo $age
```

5. **What is the root directory?** The root directory is the main directory of the system. Everything starts from here.

Symbol:

```
/
```

6. **What does “Parent Directory” mean?** The parent directory is the folder above your current folder.

Example: If you are in:

```
/home/student/Documents
```

The parent is:

```
/home/student
```

7. **What does “Current working directory” mean?** It is the directory you are currently in.

You can check it with:

```
pwd
```


8. What is an absolute path? Include an example An absolute path is the full path starting from the root directory.

Example:

```
/home/student/Documents/file.txt
```

9. What is a relative path? Include an example A relative path is a path based on your current location.

Example:

```
If you are already in /home/student:  
Documents/file.txt
```

10. What is the difference between “Your home directory” and “The home directory”? Your home directory: Your personal folder **Example:**

```
/home/student
```

The home directory: The general location where all users’ home folders are stored.

Example:

```
/home
```

For each of the following commands include: Definition, Usage/Formula, and Examples.

1. mkdir

Creates a new directory (folder) in Linux.

Usage/Formula `mkdir directory_name`

Examples `mkdir homework` → Creates a folder named homework `mkdir projects` → Creates a folder named projects

2. touch

Creates a new empty file or updates the timestamp of an existing file.

Usage/Formula `touch file_name`

Examples `touch notes.txt` → Creates an empty file named notes.txt `touch report.doc` → Creates an empty file named report.doc

3. rm

Removes (deletes) files permanently.

Usage/Formula `rm file_name`

Examples `rm notes.txt` → Deletes notes.txt `rm oldfile.doc` → Deletes oldfile.doc

4. cp

Copies files or directories from one location to another.

Usage/Formula `cp source destination`

Examples `cp file1.txt file2.txt` → Copies file1.txt into file2.txt `cp notes.txt backup.txt` → Creates a copy named backup.txt

5. mv

Moves files/directories or renames them.

Usage/Formula `mv source destination`

Examples `mv old.txt new.txt` → Renames old.txt to new.txt `mv notes.txt homework/` → Moves notes.txt into homework folder

Wildcards in Linux

Wildcards are special symbols used to match file names or groups of files.

1. Asterisk *

Definition: Matches zero or more characters.

Usage/Formula:

```
command *
```

Examples:

```
ls *.txt
```

Shows all `.txt` files.

```
rm file*
```

Deletes files starting with file.

```
cp *.jpg Pictures/
```

Copies all `.jpg` files to Pictures folder.

2. Question Mark ?

Definition: Matches exactly one character.

Usage/Formula:

```
command ?
```

Examples:

```
ls file?.txt
```

Matches `file1.txt`, `fileA.txt`

```
rm data?
```

Matches `data1`, `data2`

```
cp ?.doc Docs/
```

Matches one-letter filenames like `a.doc`

3. Brackets []

Definition: Matches one character from a list or range.

Usage/Formula:

```
command [characters]
```

Examples:

```
ls file[123].txt
```

Matches `file1.txt`, `file2.txt`, `file3.txt`

```
rm test[a-c]
```

Matches `testa`, `testb`, `testc`

```
cp report[0-9].txt Backup/
```

Matches `report1.txt`, `report5.txt`

4. Negated Brackets [!]

Definition: Matches any one character NOT listed.

Usage/Formula:

```
command [!characters]
```

Examples:

```
ls file[!1].txt
```

Matches all except `file1.txt`

```
rm data[!a-c]
```

Deletes files not ending in a, b, or c

```
cp test[!0-9].txt Folder/
```

Matches non-number endings.

Brace Expansion

Brace expansion creates many files or folders quickly.

Example 1: Create Multiple Folders

```
mkdir Week{1..5}
```

Creates:

Week1 Week2 Week3 Week4 Week5

Example 2: Create Entire Directory Structure

```
mkdir -p School/{Math,English,Science}/{Homework,Notes}
```

Creates:

```
├── Math/
│   ├── Homework
│   └── Notes
├── English/
│   ├── Homework
│   └── Notes
└── Science/
    ├── Homework
    └── Notes
```

Example 3: Create Numbered Files

```
touch file{1..3}.txt
```

Creates:

```
file2.txt
file3.txt``
```

Notes 8

Definition, Usage/Formula, and Examples.

cat

Definition: The cat command displays the contents of a file and can also combine multiple files.

Usage/Formula: `cat [options] filename`

Examples:

```
cat file.txt
cat file1.txt file2.txt
```

2.tac

Definition: The tac command displays the contents of a file in reverse order.

Usage/Formula: `tac filename`

Examples:

```
tac file.txt
```

3.head

Definition: The head command displays the first lines of a file.

Usage/Formula: `head [options] filename`

Examples:

```
head file.txt
head -5 file.txt
```

4.tail

Definition: The tail command displays the last lines of a file.

Usage/Formula: `tail [options] filename`

Examples:

```
tail file.txt  
tail -10 file.txt
```

5.cut

Definition: The cut command extracts specific columns or characters from a file.

Usage/Formula: `cut [options] filename`

Examples:

```
cut -c 1-5 file.txt  
cut -d "," -f 1 data.csv
```

6.sort

Definition: The sort command arranges lines of text in alphabetical or numerical order.

Usage/Formula: `sort [options] filename`

Examples:

```
sort names.txt  
sort -r names.txt
```

7.WC

Definition: The wc command counts lines, words, and characters in a file.

Usage/Formula: `wc [options] filename`

Examples:

```
wc file.txt  
wc -l file.txt  
wc -w file.txt
```

Notes 9

1.grep

Definition

The grep command is used to search for text or patterns inside files. It prints the lines that match the search pattern.

Usage / Formula `grep [options] pattern file`

Examples

```
Search for a word in a file
grep "root" /etc/passwd
Ignore uppercase and lowercase differences
grep -i "linux" notes.txt
Show line numbers
grep -n "error" log.txt
```

2.awk

Definition

The awk command is used to process and analyze text files. It is commonly used to display specific columns or manipulate text.

Usage / Formula `awk 'pattern {action}' file`

Examples

```
Print the first column
awk '{print $1}' names.txt
Print the first and third columns
awk '{print $1, $3}' data.txt
Print lines that contain a word
awk '/admin/ {print}' users.txt
```

3.sed

Definition

The sed command is used to search, replace, and edit text in a file or command output.

Usage / Formula `sed 'command' file`

Examples


```
Replace a word
sed 's/cat/dog/' pets.txt
Replace all occurrences in a line
sed 's/apple/orange/g' fruits.txt
Display only specific lines
sed -n '1,5p' notes.txt
```

4. Pipe (|)

Definition

The pipe operator (|) sends the output of one command directly into another command.

Usage / Formula `command1 | command2` **Examples**

```
Send cat output to grep
cat names.txt | grep "John"
Send ls output to sort
ls | sort
Count lines using wc
cat file.txt | wc -l
```

5. Redirect Output (>)

Definition

The > operator saves the output of a command into a file. If the file already exists, it will overwrite the content.

Usage / Formula `command > file`

Examples

```
Save directory list
ls > files.txt
Save users from /etc/passwd
cat /etc/passwd > users.txt
Save sorted data
sort names.txt > sorted_names.txt
```

6. Append Output (>>)

Definition

The >> operator appends the output of a command to the end of a file without deleting the existing content.

Usage / Formula `command >> file`

Examples

```
Append current date
date >> log.txt
Append directory contents
ls >> files.txt
Append system users
cat /etc/passwd >> users.txt
```

Table 1

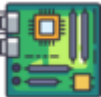
Id	Name	Email	Investments
231	Albert Master	albert.master@gmail.com	Bonds
210	Alfred Alan	aalan@gmail.com	Stocks
256	Alison Smart	asmart@biztalk.com	Residential Property
211	Ally Emery	allye@easymail.com	Stocks
248	Andrew Phips	andyp@mycorp.com	Stocks
234	Andy Mitchel	andym@hotmail.com	Stocks
226	Angus Robins	arobins@robins.com	Bonds
241	Ann Melan	ann_melan@iinet.com	Residential Property
225	Ben Bessel	benb@hotmail.com	Stocks
235	Bensen Romanoff	benr@albert.net	Bonds

Table 2

Letter Grade	Percentage
A	90 – 100%
B	80 – 89.99%
C	70 – 79.99%
D	60 – 69.99%
F	< 60%

Table 3

Part	Name
	gpu
	hard-disk

Part	Name
	motherboard
	ram